# MQTT

## Additional Exercises

Technologies for IoT Ecosystems — Software Lab

---

## Introduction

The following four exercises introduce new, self-contained MQTT scenarios that build on the publish/subscribe fundamentals covered in the course. Each exercise is independent and introduces a distinct real-world use case. All exercises use the MyMQTT helper class pattern shown in the course material and the Eclipse Foundation public broker (iot.eclipse.org, port 1883). Replace 'groupXX' in every topic with your own group identifier to avoid collisions with other teams.

### MyMQTT Reference (from course material)

All exercises must be implemented using OOP classes. Use the MyMQTT wrapper below as the MQTT communication layer:

```
import paho.mqtt.client as PahoMQTT

class MyMQTT:
    def __init__(self, clientID, broker, port, notifier):
        self.broker   = broker
        self.port     = port
        self.notifier = notifier
        self._paho_mqtt = PahoMQTT.Client(clientID, False)
        self._paho_mqtt.on_connect = self.myOnConnect
        self._paho_mqtt.on_message = self.myOnMessageReceived

    def myOnConnect(self, paho_mqtt, userdata, flags, rc):
        print('Connected to %s with result code: %d' % (self.broker, rc))

    def myOnMessageReceived(self, paho_mqtt, userdata, msg):
        self.notifier.notify(msg.topic, msg.payload)

    def myPublish(self, topic, msg):
        self._paho_mqtt.publish(topic, msg, 2)

    def mySubscribe(self, topic):
        self._paho_mqtt.subscribe(topic, 2)

    def start(self):
        self._paho_mqtt.connect(self.broker, self.port)
        self._paho_mqtt.loop_start()
```

```
    def stop(self):
        self._paho_mqtt.loop_stop()
        self._paho_mqtt.disconnect()
```

## Exercise 1 — Smart Parking Lot Monitor

A parking IoT has 6 slots (slot_1 … slot_6). Each slot is equipped with a presence sensor that publishes its status (free or occupied) on a dedicated MQTT topic. A monitor client subscribes to all slot topics and maintains a real-time view of the lot. The monitor must print the full status of the lot every time any slot changes, and must publish the total number of free slots on a summary topic after every update.

**Requirements**

1. Create a SlotSensor class. Each instance simulates one parking slot: it toggles between 'free' and 'occupied' at a random interval between 5 and 15 seconds and publishes its current status on the topic tiot/groupXX/parking/<slotID>. Payload: {"slot": "<slotID>", "status": "free"|"occupied"}.
2. Create a ParkingMonitor class. It subscribes to tiot/groupXX/parking/# (wildcard). Each time a message arrives, it updates the internal state dict and prints the full parking lot status (one line per slot).
3. After every update, the monitor must publish the count of free slots on the topic tiot/groupXX/parking/summary. Payload: {"free_slots": <int>, "total_slots": 6}.
4. Both SlotSensor and ParkingMonitor must use the MyMQTT class.
5. Run 6 SlotSensor instances (one per slot) and one ParkingMonitor in separate terminals.

**Example**

**Expected interaction / output:**

```
$ python exercise_1.py monitor
Connected to iot.eclipse.org with result code: 0
subscribing to tiot/groupXX/parking/#

--- Parking lot update ---
slot_1: occupied
slot_2: free
slot_3: free
slot_4: occupied
slot_5: free
slot_6: free
Free slots: 4 / 6
--------------------------
```

**Tips & Notes**

⚠ Use random.randint(5, 15) to set each sensor's toggle interval.

⚠ Parse the slot ID from the MQTT topic: topic.split('/')[-1].

⚠ The summary topic tiot/groupXX/parking/summary will also be received by the wildcard subscription — filter it out in the monitor's notify() method.

⚠ Use time.sleep() inside a loop in each SlotSensor to control the toggle interval.

## Exercise 2 — Remote Device Configuration via MQTT and REST

A remote IoT device exposes two interfaces: a CherryPy REST API to read its current configuration, and an MQTT subscription to receive configuration updates. A separate configuration client can push new settings either by publishing directly on the MQTT config topic or by sending a PUT request to the REST API. Whenever the device configuration changes, the device logs the change to the console and publishes a confirmation message on an ack topic.

**Requirements**

1. Create a RemoteDevice class with an internal configuration dict: {"sampling_rate": 10, "active": true, "location": "room_A"}. The device subscribes to topic tiot/groupXX/device/config and listens for JSON payloads containing one or more fields to update.
2. Expose a REST endpoint (CherryPy, MethodDispatcher): GET /device/config → returns the full current configuration as JSON. PUT /device/config with a JSON body → applies the updates, logs the change, and publishes the updated config on tiot/groupXX/device/ack.
3. When a configuration update arrives via MQTT, the device must apply it, log it to the console, and publish the updated configuration on tiot/groupXX/device/ack.
4. Create a ConfigClient class that uses MyMQTT to subscribe to the ack topic and print every acknowledgement received. Run it in a separate terminal.
5. Both RemoteDevice and ConfigClient must use the MyMQTT class.

**Example**

**Expected interaction / output:**

```
$ curl http://localhost:8080/device/config
{"sampling_rate": 10, "active": true, "location":
"room_A"}

$ curl -X PUT http://localhost:8080/device/config \
       -H 'Content-Type: application/json' \
       -d '{"sampling_rate": 5, "location": "room_B"}'
{"sampling_rate": 5, "active": true, "location":
"room_B"}

# ConfigClient terminal:
ACK received — new config: {"sampling_rate": 5,
"active": true, "location": "room_B"}
```

**Tips & Notes**

⚠ Start the MQTT loop with loop_start() (non-blocking) so CherryPy can also run.

⚠ Use threading.Lock() to protect the config dict from concurrent REST and MQTT access.

⚠ The ConfigClient only needs to subscribe to the ack topic — it does not publish anything.

## Exercise 3 — Public Transport Arrival Board

A city has 3 bus lines (line_1, line_2, line_3), each with 4 stops (stop_A, stop_B, stop_C, stop_D). Each bus line has a simulated bus that moves through the stops in order and publishes its current stop on a structured topic. A central arrival board subscribes to all bus topics using wildcards and, based on the user's choice, displays the arrival information for a specific stop or a specific line. The user can change the view at any time without restarting the program.

**Requirements**

1. Create a BusSimulator class. Each instance represents one bus on one line. The bus cycles through stops stop_A → stop_B → stop_C → stop_D → stop_A → … with a random wait of 4–8 seconds at each stop. It publishes on topic tiot/groupXX/transport/<lineID>/<stopID>. Payload: {"line": "<lineID>", "stop": "<stopID>", "timestamp": <epoch>}.
2. Create an ArrivalBoard class. It subscribes to tiot/groupXX/transport/# (wildcard) and maintains a state dict: {line → current_stop}.
3. The board must offer an interactive menu (like Exercise 3 in the course material): l — show all arrivals for a specific line (user types line ID); s — show all lines currently at or heading to a specific stop; a — show the current position of all lines; c — return to the main menu; q — quit.
4. When a relevant update arrives (for the currently selected view), the board must automatically reprint the current information.
5. Both BusSimulator and ArrivalBoard must use the MyMQTT class. Run 3 BusSimulator instances and one ArrivalBoard in separate terminals.

**Example**

**Expected interaction / output:**

```
$ python exercise_3.py board
Connected to iot.eclipse.org with result code: 0
subscribing to tiot/groupXX/transport/#

What do you want to see?
  l: arrivals for a specific line
  s: lines at a specific stop
  a: all lines
  c: back to this menu
  q: quit
a
line_1 → stop_B
line_2 → stop_D
line_3 → stop_A
c
l
Type the line ID [line_1 / line_2 / line_3]: line_2
line_2 is currently at stop_D
[update] line_2 is now at stop_A
```

**Tips & Notes**

⚠ Parse line and stop from the topic: topic.split('/')[-2] gives the line, topic.split('/')[-1] gives the stop.

⚠ Use a threading.Event or a simple flag to know which view is currently active, so that notify() can decide whether to reprint.

⚠ Use input() in the main thread and let the MQTT loop run in the background via loop_start().

## Exercise 4 — Shared Whiteboard

Two or more users share a virtual whiteboard over MQTT. Each user can add a text note to the whiteboard by publishing on a notes topic. All clients subscribe to the same topic and keep a local copy of the whiteboard in sync. A user can also delete one of their own notes by publishing a delete command. The whiteboard must enforce the rule that a user can only delete notes they created themselves — attempts to delete someone else's note must be rejected and logged.

**Requirements**

1. Create a WhiteboardClient class. Each instance represents one user. It subscribes to tiot/groupXX/whiteboard/notes (all add/delete events).
2. To add a note, the client publishes on tiot/groupXX/whiteboard/notes. Payload: {"action": "add", "note_id": "<uuid>", "author": "<user>", "text": "<content>", "timestamp": <epoch>}.
3. To delete a note, the client publishes on the same topic. Payload: {"action": "delete", "note_id": "<uuid>", "author": "<user>"}. On receiving a delete event, each client must verify that author matches the note's original author before removing it from the local board; otherwise, print a rejection message.
4. Each client must offer an interactive menu: a — add a new note (prompt for text); d — delete one of your own notes (show your notes and prompt for note_id); s — show the current whiteboard (all notes, sorted by timestamp); q — quit.
5. Both add and delete events are received by all clients (including the sender), so the local state must be updated from the incoming MQTT message, not from the local action directly. Use the MyMQTT class.

**Example**

**Expected interaction / output:**

```
$ python exercise_4.py Alice
Connected. Welcome, Alice!

  a: add a note
  d: delete one of your notes
  s: show whiteboard
  q: quit
a
Note text: Hello from Alice!
[note added] Alice: 'Hello from Alice!' (id: a1b2c3)
s
--- Whiteboard ---
[a1b2c3] Alice (10:02:01): Hello from Alice!
[d4e5f6] Bob   (10:02:45): Hi Alice!
------------------
d
Your notes:
  [a1b2c3] Hello from Alice!
Note ID to delete: d4e5f6
[rejected] You can only delete your own notes.
```

**Tips & Notes**

⚠ Use uuid.uuid4().hex[:6] to generate short unique note IDs.

⚠ Store notes in a dict keyed by note_id: {note_id: {author, text, timestamp}}.

| | |
|---|---|
| | ⚠ Since all clients receive all events (including the sender's own), do NOT update the local state in the publish path — wait for the MQTT message to come back instead.<br>⚠ Use input() in the main thread; the MQTT loop runs in the background via loop_start(). |